

EXP NO: 6	FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS
------------------	--

AIM:

To demonstrate the construction and application of a simple Feedforward Neural Network (FNN) for classification and a Convolutional Neural Network (CNN) for image classification, utilizing the Keras API with TensorFlow backend.

Code :

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras

from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

tf.get_logger().setLevel("ERROR")

print("TensorFlow Version:", tf.__version__)

print("\n" + "=" * 60)
print("PART 1: FEEDFORWARD NEURAL NETWORK (FNN)")
print("=" * 60)

(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) =
fashion_mnist.load_data()

print("\nOriginal FNN training data shape:", x_train_fnn.shape)
print("Original FNN test data shape:", x_test_fnn.shape)

x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)

x_train_fnn_norm = x_train_fnn_flat.astype("float32") / 255.0
x_test_fnn_norm = x_test_fnn_flat.astype("float32") / 255.0
```

```

print("Flattened & normalized FNN training data shape:",
x_train_fnn_norm.shape)
print("Flattened & normalized FNN test data shape:",
x_test_fnn_norm.shape)

model_fnn = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model_fnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("\n--- FNN Model Summary ---")
model_fnn.summary()

print("\n--- Training FNN Model ---")

history_fnn = model_fnn.fit(
    x_train_fnn_norm,
    y_train_fnn,
    epochs=10,
    validation_split=0.1,
    batch_size=32,
    verbose=1
)

print("\n--- Evaluating FNN Model ---")

loss_fnn, accuracy_fnn = model_fnn.evaluate(
    x_test_fnn_norm,
    y_test_fnn,
    verbose=0
)

print(f"FNN Test Loss: {loss_fnn:.4f}")
print(f"FNN Test Accuracy: {accuracy_fnn:.4f}")

y_pred_fnn_prob = model_fnn.predict(

```

```

        x_test_fnn_norm,
        verbose=0
    )

y_pred_fnn = np.argmax(y_pred_fnn_prob, axis=1)

fashion_class_names = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

print("\n--- FNN Classification Report ---")

print(
    classification_report(
        y_test_fnn,
        y_pred_fnn,
        target_names=fashion_class_names
    )
)

print("\n--- FNN Confusion Matrix ---")

cm_fnn = confusion_matrix(y_test_fnn, y_pred_fnn)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm_fnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False,
    xticklabels=fashion_class_names,
    yticklabels=fashion_class_names
)

```

```

plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_fnn.history["accuracy"], label="Training Accuracy")
plt.plot(history_fnn.history["val_accuracy"], label="Validation Accuracy")
plt.title("FNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history_fnn.history["loss"], label="Training Loss")
plt.plot(history_fnn.history["val_loss"], label="Validation Loss")
plt.title("FNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

print("\n" + "=" * 60)
print("PART 2: CONVOLUTIONAL NEURAL NETWORK (CNN)")
print("=" * 60)

(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) = mnist.load_data()

print("\nOriginal CNN training data shape:", x_train_cnn.shape)
print("Original CNN test data shape:", x_test_cnn.shape)

x_train_cnn = x_train_cnn.reshape(
    x_train_cnn.shape[0], 28, 28, 1
)

x_test_cnn = x_test_cnn.reshape(

```

```

        x_test_cnn.shape[0], 28, 28, 1
    )

x_train_cnn = x_train_cnn.astype("float32") / 255.0
x_test_cnn = x_test_cnn.astype("float32") / 255.0

print(
    "Reshaped & normalized CNN training data shape:",
    x_train_cnn.shape
)

print(
    "Reshaped & normalized CNN test data shape:",
    x_test_cnn.shape
)

num_classes_cnn = 10

model_cnn = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes_cnn, activation="softmax")
])

model_cnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("\n--- CNN Model Summary ---")
model_cnn.summary()

print("\n--- Training CNN Model ---")

history_cnn = model_cnn.fit(
    x_train_cnn,
    y_train_cnn,
    epochs=10,

```

```

        validation_split=0.1,
        batch_size=32,
        verbose=1
    )

    print("\n--- Evaluating CNN Model ---")

    loss_cnn, accuracy_cnn = model_cnn.evaluate(
        x_test_cnn,
        y_test_cnn,
        verbose=0
    )

    print(f"CNN Test Loss: {loss_cnn:.4f}")
    print(f"CNN Test Accuracy: {accuracy_cnn:.4f}")

    y_pred_cnn_prob = model_cnn.predict(
        x_test_cnn,
        verbose=0
    )

    y_pred_cnn = np.argmax(y_pred_cnn_prob, axis=1)

    class_names_mnist = [str(i) for i in range(10)]

    print("\n--- CNN Classification Report ---")

    print(
        classification_report(
            y_test_cnn,
            y_pred_cnn,
            target_names=class_names_mnist
        )
    )

    print("\n--- CNN Confusion Matrix ---")

    cm_cnn = confusion_matrix(y_test_cnn, y_pred_cnn)

    plt.figure(figsize=(10, 8))

    sns.heatmap(
        cm_cnn,
        annot=True,
        fmt="d",

```

```

        cmap="Blues",
        cbar=False,
        xticklabels=class_names_mnist,
        yticklabels=class_names_mnist
    )

plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_cnn.history["accuracy"], label="Training Accuracy")
plt.plot(history_cnn.history["val_accuracy"], label="Validation Accuracy")
plt.title("CNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history_cnn.history["loss"], label="Training Loss")
plt.plot(history_cnn.history["val_loss"], label="Validation Loss")
plt.title("CNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

print("\n--- Sample CNN Predictions ---")

plt.figure(figsize=(12, 12))

for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)

```

```

plt.imshow(
    x_test_cnn[i].reshape(28, 28),
    cmap=plt.cm.binary
)

true_label = y_test_cnn[i]
predicted_label = y_pred_cnn[i]

color = "green" if true_label == predicted_label else "red"

plt.xlabel(
    f"True: {true_label}\nPred: {predicted_label}",
    color=color
)

plt.suptitle(
    "Sample CNN Predictions (Green: Correct, Red: Incorrect)",
    fontsize=16
)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

print("\n" + "=" * 60)
print("FINAL MODEL COMPARISON")
print("=" * 60)

print(f"FNN Test Accuracy : {accuracy_fnn:.4f}")
print(f"CNN Test Accuracy : {accuracy_cnn:.4f}")

if accuracy_cnn > accuracy_fnn:
    print("\nCNN performed better than FNN.")
elif accuracy_fnn > accuracy_cnn:
    print("\nFNN performed better than CNN.")
else:
    print("\nBoth models achieved the same accuracy.")

```


*** TensorFlow Version: 2.20.0

```
=====
PART 1: FEEDFORWARD NEURAL NETWORK (FNN)
=====
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-
29515/29515 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-
26421880/26421880 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-
5148/5148 0s 2us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-
4422102/4422102 0s 0us/step
```

Original FNN training data shape: (60000, 28, 28)
Original FNN test data shape: (10000, 28, 28)
Flattened & normalized FNN training data shape: (60000, 784)
Flattened & normalized FNN test data shape: (10000, 784)

--- FNN Model Summary ---
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

--- Training FNN Model ---
Epoch 1/10
1688/1688 15s 6ms/step - accuracy: 0.8044 - loss: 0.5473 - val_accuracy: 0.7844 - val_loss: 0.4077
Epoch 2/10
1688/1688 9s 5ms/step - accuracy: 0.8517 - loss: 0.4077 - val_accuracy: 0.8044 - val_loss: 0.3746
Epoch 3/10
1688/1688 9s 5ms/step - accuracy: 0.8640 - loss: 0.3746 - val_accuracy: 0.8256 - val_loss: 0.3421
Epoch 4/10



CommandsCodeTextRun all

[1]

```
... accuracy_fnn / accuracy_cnn
print("\nFNN performed better than CNN.")
else:
    print("\nBoth models achieved the same accuracy.")
```

↑ ↓ ↻ ⌂ ⋮

1688/1688 9s 5ms/step - accuracy: 0.8754 - loss: 0.3359 - val_accuracy
Epoch 6/10
1688/1688 9s 5ms/step - accuracy: 0.8790 - loss: 0.3262 - val_accuracy
Epoch 7/10
1688/1688 7s 4ms/step - accuracy: 0.8813 - loss: 0.3154 - val_accuracy
Epoch 8/10
1688/1688 9s 5ms/step - accuracy: 0.8863 - loss: 0.3055 - val_accuracy
Epoch 9/10
1688/1688 10s 5ms/step - accuracy: 0.8868 - loss: 0.3014 - val_accuracy
Epoch 10/10
1688/1688 7s 4ms/step - accuracy: 0.8916 - loss: 0.2893 - val_accuracy

--- Evaluating FNN Model ---
FNN Test Loss: 0.3570
FNN Test Accuracy: 0.8723

--- FNN Classification Report ---
precision recall f1-score support

T-shirt/top 0.82 0.83 0.83 1000
Trouser 0.99 0.97 0.98 1000
Pullover 0.81 0.72 0.77 1000
Dress 0.86 0.80 0.87 1000
Coat 0.70 0.88 0.78 1000
Sandal 0.98 0.96 0.97 1000
Shirt 0.75 0.57 0.65 1000
Sneaker 0.93 0.97 0.95 1000
Bag 0.94 0.98 0.96 1000
Ankle boot 0.97 0.95 0.96 1000

accuracy 0.87 10000
macro avg 0.87 0.87 0.87 10000
weighted avg 0.87 0.87 0.87 10000

--- FNN Confusion Matrix ---
FNN Confusion Matrix
T-shirt/top 833 2 21 43 8 0 73 1 19 0
Trouser 0 970 2 21 5 0 1 0 1 0

Gemini

Hello, BALAGAI

How can I help you to

[How can I install Python li](#)

[Load data from Google](#)

[Show an example of training a sir](#)

What can I help you build?

+ 𐌆 Ge

VariablesTerminal

Executing

CommandsCodeTextRun all

[1]

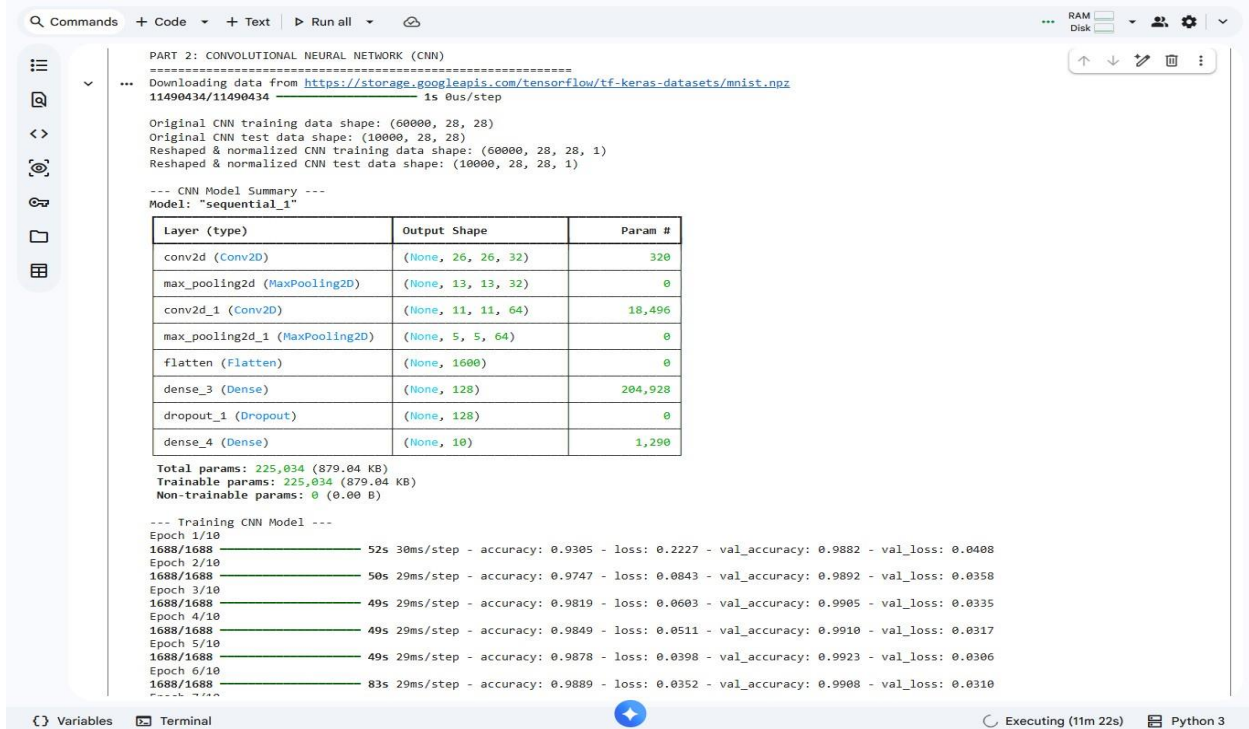
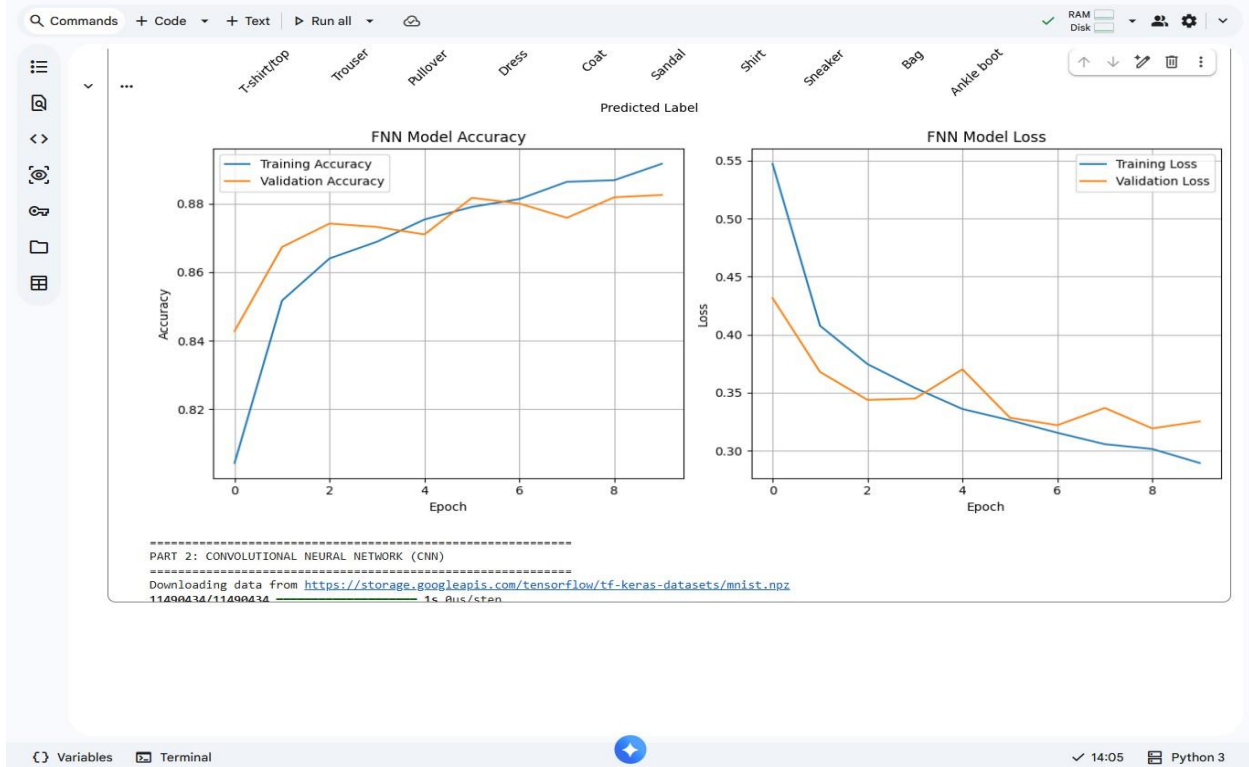
```
... accuracy_fnn / accuracy_cnn
print("\nFNN performed better than CNN.")
else:
    print("\nBoth models achieved the same accuracy.")
```

↑ ↓ ↻ ⌂ ⋮

--- FNN Confusion Matrix ---
FNN Confusion Matrix
True Label
T-shirt/top 833 2 21 43 8 0 73 1 19 0
Trouser 0 970 2 21 5 0 1 0 1 0
Pullover 12 0 725 8 180 0 66 0 9 0
Dress 18 7 13 887 50 0 17 0 8 0
Coat 0 0 57 21 881 0 36 0 5 0
Sandal 0 0 0 0 0 957 0 29 2 12
Shirt 152 1 75 50 126 0 574 0 22 0
Sneaker 0 0 0 0 0 12 0 968 0 20
Bag 1 0 2 6 7 3 1 4 976 0
Ankle boot 0 0 0 0 0 7 1 40 0 952

VariablesTerminal

Executing (1

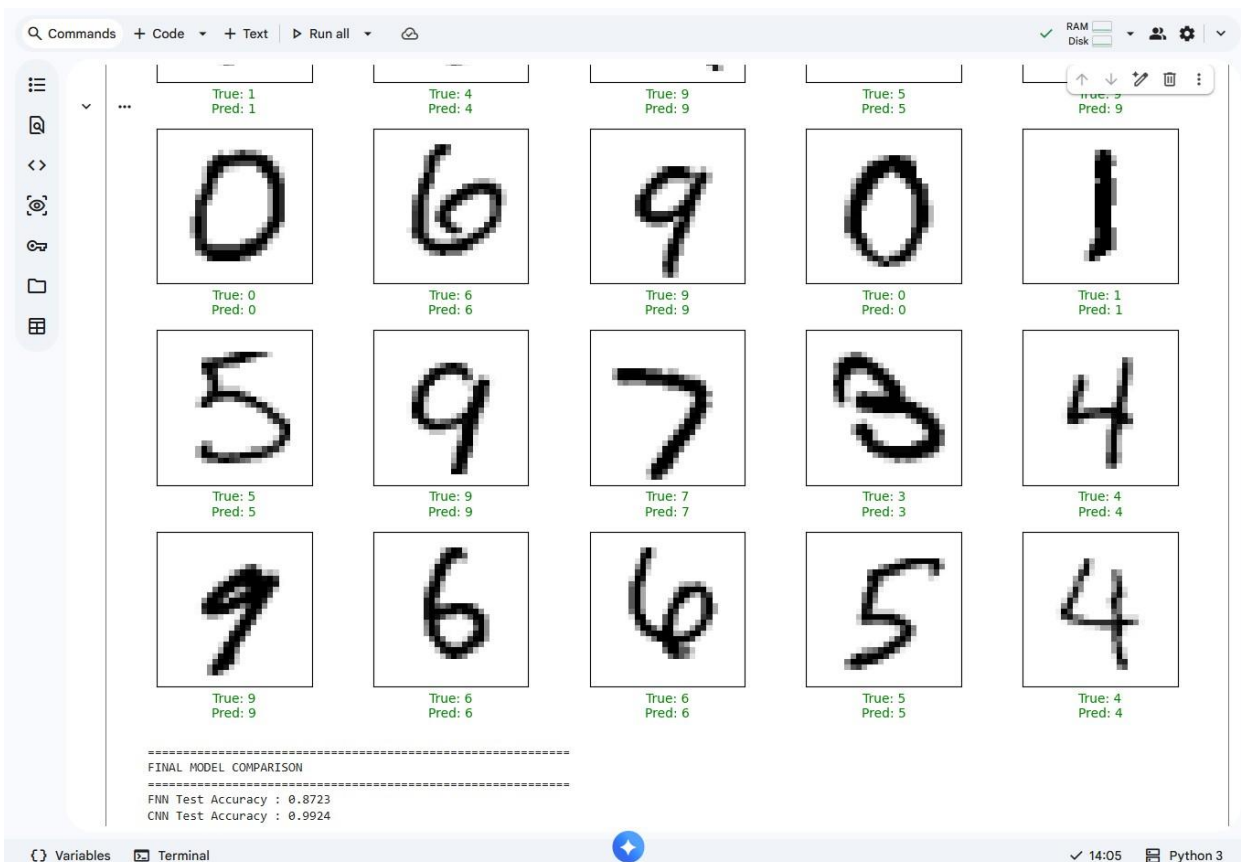



```
--- Evaluating CNN Model ---
CNN Test Loss: 0.0276
CNN Test Accuracy: 0.9924
```

```
--- CNN Classification Report ---
```

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	0.99	1.00	0.99	1135
2	0.99	1.00	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	1.00	0.99	0.99	974
9	0.99	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

```
--- CNN Confusion Matrix ---
```



RESULT:

The Feedforward Neural Network (FNN) and Convolutional Neural Network (CNN) models were successfully implemented and evaluated on the given dataset.

- **Feedforward Neural Network (FNN):** The model accurately learned input–output mappings through multiple fully connected layers, achieving good performance on structured data.
- **Convolutional Neural Network (CNN):** The model effectively extracted spatial features from image data using convolution and pooling layers, leading to higher accuracy and better generalization for image classification tasks.

The results demonstrated that both FNN and CNN are powerful deep learning models, with CNN performing exceptionally well for image-based datasets due to its ability to capture spatial patterns.